# Verification of "Bridging the Gap between Constant Step Size SGD and Markov Chains"

**Reference:** Dieuleveut, A., Durmus, A., & Bach, F. (2017). arXiv:1707.06386

## Project Abstract

This research project experimentally verifies the asymptotic properties of **Constant Step-Size Stochastic Gradient Descent (SGD)**. By reinterpreting SGD as a **homogeneous Markov Chain**, we explore the convergence of Polyak-Ruppert averaged iterates to a stationary distribution $\pi_\gamma$. We demonstrate that while averaging reduces variance, it exposes an $O(\gamma)$ asymptotic bias caused by the non-quadratic nature of the loss landscape. Finally, we prove that **Richardson-Romberg (RR) extrapolation** effectively eliminates this bias, achieving a high-accuracy estimate of the global optimum $\theta^*$.

---

## Theoretical Framework: SGD as an MCMC Process

Unlike standard SGD where the learning rate $\gamma_k \to 0$, keeping $\gamma$ constant prevents the algorithm from settling at a single point. Instead, it behaves as a Markov Chain Monte Carlo (MCMC) sampler.

1. **The Markov Property & AR(1) Dynamics:** The update rule for SGD is:

$$\theta_{k+1} = \theta_k - \gamma \nabla f_{i_k}(\theta_k)$$
$$\nabla f_{i_k}(\theta_k) = \nabla f(\theta_k) + \epsilon_k$$

   where $\nabla f_{i_k}$ is the stochastic gradient evaluated on a single, randomly sampled data point. Noise $\epsilon_k$ is considered unknown but zero mean/stationary. Because the next state $\theta_{k+1}$ depends *only* on the current state $\theta_k$ and a random variable, the sequence of iterates $(\theta_k)_{k\geq 0}$

   forms a **homogeneous Markov Chain**. This is because the strong convexity of the loss function (ensured by L2 regularization), allows the gradient to be approximated as linear with respect to $\theta_k$. When this linear gradient is substituted into the SGD update rule, the sequence of iterates $(\theta_k)_{k\geq 0}$ forms an Autoregressive process of order 1 (AR(1)), which is a type of homogeneous Markov Chain.

   The update rule $\theta_{k+1} = \theta_k - \gamma \nabla f_{i_k}(\theta_k)$ depends only on the current state and a random variable - new datapoint. Near the optimum $\theta^*$, the strongly convex objective $f(\theta)$ allows for a linear approximation of the gradient:

$$\nabla f(\theta_k) \approx H(\theta^\star)(\theta_k - \theta^\star)$$

This transforms the SGD iterates into a **Vector Autoregressive Process (AR(1))**:

$$\delta_{k+1} \approx (I - \gamma H(\theta^*))\delta_k - \gamma\epsilon_k$$

where $\delta_k = \theta_k - \theta^*$.

2. **Ergodicity and Invariant Measures:** Under the stability condition $\gamma < 2/L$, the chain is **ergodic**. This ensures that the time average (Polyak-Ruppert average)

$$\bar{\theta}_k = \frac{1}{k}\sum \theta_i$$

converges to the mean of a unique stationary distribution $\pi_\gamma$.

3. **The Asymptotic Bias Challenge:** In a non-quadratic landscape, the 'bowl' is not symmetric. The interaction between the constant step-size $\gamma$ and the zero`-mean noise $\epsilon_k$ results in an **asymmetric stationary distribution** $\pi_\gamma$.

The mean of this distribution $\bar{\theta}_\gamma$ is shifted from the true optimum:

$$\bar{\theta}_\gamma = \theta^* + \gamma\Delta + O(\gamma^2)$$

where $\Delta$ is the asymptotic bias. Richardson-Romberg extrapolation uses parallel chains ($2\bar{\theta}_k^{(\gamma)} - \bar{\theta}_k^{(2\gamma)}$) to mathematically cancel the $\gamma\Delta$ term, isolating $\theta^*$.

4. **The Solution: Richardson-Romberg Extrapolation:** By running parallel chains with different step sizes ($\gamma$ and $2\gamma$), we can cancel the first-order bias term $\gamma\Delta$:

$$\theta_{RR} = 2\bar{\theta}_k^{(\gamma)} - \bar{\theta}_k^{(2\gamma)} \approx \theta^* + O(\gamma^2)$$

---

## Notebook Objectives

In the cells below, we construct a Synthetic Logistic Regression environment to experimentally prove this theory. The pipeline includes:

1. **Convergence Analysis (Log-Log Plot):** Demonstrating that the RR estimator achieves a lower Mean Squared Error (MSE) floor than standard averaged SGD.
2. **Distribution Analysis:** Visualizing the high-variance raw iterates $\theta_k$ versus the low-variance averaged iterates $\bar{\theta}_k$ in various snapshots throughout training.
3. **MCMC Stationary Measure Animation:** Animating the histograms of the

parallel chains to visualize the $O(\gamma)$ bias and how the RR extrapolation successfully centers on the Ground Truth $\theta^*$.

# Implementation Strategy: SGD as a Transition Kernel

## 1. Markovian Design

The `SGDExperiment` class implements the **Markov Transition Kernel**. By maintaining the current iterate $\theta_k$ and the running Polyak-Ruppert average $\bar{\theta}_k$, it allows us to track the evolution of the invariant measure $\pi_\gamma$ in real-time. Through this class we can **generate data**, **calculate gradient**, and **update** the model and the average count.

### Data Generation Strategy

To simulate a realistic non-quadratic environment, the data generation follows a controlled probabilistic process:

1. **Ground Truth (theta_star)**: We define a fixed target vector $\theta^* \sim \mathcal{N}(1, 0.1)$. This serves as the 'Hidden Truth' our algorithm must discover.
2. **Feature Sampling (X)**: Inputs are drawn from a uniform distribution $X \sim U(-1, 1)$. This provides bounded support, satisfying Assumption A3 of the theoretical framework.
3. **Labeling (Y)**: Labels are generated using the Sigmoid probability: $P(Y = 1|x) = 1/(1 + e^{-x^T\theta^*})$. By drawing binary outcomes $Y \in \{-1, 1\}$ from this probability, we create a stochastic relationship between features and labels.
4. **The Non-Quadratic Result**: This specific setup ensures that the second derivative of the loss (the Hessian) is $p(1-p)xx^T$, which varies with $\theta$. This creates the 'warped' curvature required to demonstrate the $O(\gamma)$ asymptotic bias.

## 2. Symmetry & Initialization

Iterates are initialized via $\mathcal{N}(0, 0.1)$ to break symmetry. This ensures the chain explores the non-quadratic landscape from an arbitrary starting point, revealing the true relaxation dynamics toward the biased stationary mean.

## 3. The Bias Mechanism

Unlike linear models, our Logistic Regression features a **position-dependent Hessian**. The interaction between a constant step $\gamma$ and this varying curvature is the primary trigger for the $O(\gamma)$ bias. Running parallel chains on the same data stream allows us to isolate this bias from stochastic variance.

## 4. Convergence Analysis

We utilize **Log-Log plots** of $\|\bar{\theta}_k - \theta^*\|^2$. This transformation visualizes the $O(1/k)$ variance decay as a linear slope, which eventually plateaus at the 'deterministic floor' created by the asymptotic bias.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from IPython.display import HTML

# Helper function for sigmoid
def sigmoid(x: np.ndarray) -> np.ndarray:
    return 1 / (1 + np.exp(-x))

# Class to manage the SGD experiment for a single chain
class SGDExperiment:
    def __init__(self, d, gamma, theta_star, mu=0.01, burn_in_period=0):
        self.d = d
        self.gamma = gamma
        self.gamma_n_initial = gamma # Store initial gamma for sqrt_n ste
        self.theta_star = theta_star
        self.mu = mu # L2 regularization parameter
        self.burn_in_period = burn_in_period

        # Initialize iterate and averaged iterate
        self.theta_k = 0.1 * np.random.randn(d) # Initial random guess fo
        self.theta_bar_k = np.copy(self.theta_k)

        self.k = 0 # Iteration counter
        self.averaged_k = 0 # Counter for actual averaging steps (after b

        # For storing historical data for plots/animation
        self.theta_history = []
        self.theta_bar_history = []

    # Generate a single sample (x, y)
    def generate_sample(self):
        x = np.random.uniform(-1, 1, self.d)
        prob_y_is_1 = sigmoid(np.dot(x, self.theta_star))
        y = 2 * (np.random.rand() < prob_y_is_1) - 1 # y in {-1, 1}
        return x, y

    # Calculate the gradient of the logistic loss with L2 regularization
    def calculate_gradient(self, x, y):
        pred_prob = sigmoid(np.dot(x, self.theta_k))
        # Logistic loss for Y in {-1, 1}: L(y, h) = log(1 + exp(-y * h))
        # Gradient: -y*x / (1 + exp(y * np.dot(x, self.theta_k))) + mu *
        grad_val = -y * x / (1 + np.exp(y * np.dot(x, self.theta_k))) + s
        return grad_val

    # Perform one SGD step and update averaged iterate with burn-in
```

```
    # Perform one SGD step and update averaged iterate with burn-in
    def update(self, x, y):
        grad = self.calculate_gradient(x, y)
        self.theta_k = self.theta_k - self.gamma * grad

        self.k += 1
        # Polyak-Ruppert averaging (iterative update) only after burn-in
        if self.k > self.burn_in_period:
            self.averaged_k += 1
            if self.averaged_k == 1:
                self.theta_bar_k = np.copy(self.theta_k)
            else:
                self.theta_bar_k = ((self.averaged_k - 1) * self.theta_ba

    def update_sqrt_n(self, x, y):
        # Calculate current step size based on sqrt(k+1)
        current_gamma_n = self.gamma_n_initial / np.sqrt(self.k + 1)

        grad = self.calculate_gradient(x, y)
        self.theta_k = self.theta_k - current_gamma_n * grad

        self.k += 1
        # Polyak-Ruppert averaging (iterative update) only after burn-in
        if self.k > self.burn_in_period:
            self.averaged_k += 1
            if self.averaged_k == 1:
                self.theta_bar_k = np.copy(self.theta_k)
            else:
                self.theta_bar_k = ((self.averaged_k - 1) * self.theta_ba

    def store_history(self):
        self.theta_history.append(np.copy(self.theta_k))
        # Only store theta_bar_k if averaging has started
        if self.averaged_k > 0:
            self.theta_bar_history.append(np.copy(self.theta_bar_k))
        else:
            # If no averaging yet, append a placeholder (e.g., last theta
            # For consistency in history length with other chains, append
            self.theta_bar_history.append(np.copy(self.theta_k))

    # Reset for new simulation runs if needed
    def reset(self):
        self.theta_k = np.random.randn(self.d)
        self.theta_bar_k = np.copy(self.theta_k)
        self.k = 0
        self.averaged_k = 0
        self.theta_history = []
        self.theta_bar_history = []
```

## Experimental Cases & Comparative Baselines

To prove the theoretical claims, we simulate and compare the following six cases:

To prove the theoretical claims, we simulate and compare the following six cases:

1. **Standard Constant SGD ($\gamma, 2\gamma, 4\gamma$):** These serve as our primary MCMC samplers. They exhibit the $O(1/k)$ variance reduction but plateau at the $O(\gamma)$ bias floor.
2. **Decreasing Step-Size SGD ($2\gamma/\sqrt{k}$):** A standard optimization baseline. While it eventually eliminates bias, its convergence is significantly slower than constant-step SGD, illustrating the 'speed vs. accuracy' trade-off.
3. **Richardson-Romberg (RR) Extrapolation:** Our primary solution. It combines the $\gamma$ and $2\gamma$ chains to cancel the first-order bias, aiming for $O(\gamma^2)$ accuracy.
4. **Triple Richardson-Romberg (3RR):** An advanced case using $\gamma$, $2\gamma$, and $4\gamma$. It aims to eliminate even higher-order bias terms ($O(\gamma^3)$), though often at the cost of increased stochastic variance.

```
# --- Simulation Parameters ---
n_iterations = 2_000_000
burn_in_period = 0 # 5_000 # Number of steps before collecting data for his
d = 4
mu = 0.0001 # L2 regularization parameter - strong convexity
gamma_base = 3.0 / (4*d) # Base step size - the greatest step (4*gamma) nee
                         # 3/4d was chosen over 1/4d due to its faster conv
# True parameter theta_star
# np.random.seed(10) # for reproducibility - COMMENTED-OUT though as fully
theta_star = np.random.normal(1, 0.1, d) # N(1,0.1)
print("theta_star = ", theta_star)

# Instantiate two SGDExperiment objects
sgd_gamma_chain = SGDExperiment(d, gamma_base, theta_star, mu, burn_in_peri
sgd_2gamma_chain = SGDExperiment(d, 2 * gamma_base, theta_star, mu, burn_ir
sgd_4gamma_chain = SGDExperiment(d, 4 * gamma_base, theta_star, mu, burn_ir
sgd_2gamma_sqrtn_chain = SGDExperiment(d, 2 * gamma_base, theta_star, mu, b
# sgd_4gamma_sqrtn_chain = SGDExperiment(d, 4 * gamma_base, theta_star, mu,

# Lists to store results for plotting
loss_history_gamma = []
loss_history_2gamma = []
loss_history_4gamma = []
loss_history_2gamma_sqrtn = []
# loss_history_4gamma_sqrtn = []

theta_bar_norm_history_gamma = []
theta_bar_norm_history_2gamma = []
theta_bar_norm_history_4gamma = []
theta_bar_norm_history_2gamma_sqrtn = []
# theta_bar_norm_history_4gamma_sqrtn = []
theta_bar_norm_history_rr = []
theta_bar_norm_history_3rr = []

# Histories to store actual vectors for distribution plots after burn-in
theta_bar_RR_dist_history = []
```

```
theta_bar_RR_dist_history = []
theta_bar_3RR_dist_history = []
theta_RR_dist_history = [] # Even though  RR does not deal with raw data,
                           # a history variable is alocated for testing pur
theta_3RR_dist_history = [] # Same applies here

theta_k_history_for_hist = [] # For bias analysis (after burn-in)
theta_bar_k_history_for_hist = [] # For bias analysis (after burn-in)

# --- Simulation Loop ---
print("Starting simulation...")
for i in range(n_iterations):
    x_i, y_i = sgd_gamma_chain.generate_sample() # use one generator for bo

    # Update all chains
    sgd_gamma_chain.update(x_i, y_i)
    sgd_2gamma_chain.update(x_i, y_i)
    sgd_4gamma_chain.update(x_i, y_i)
    sgd_2gamma_sqrtn_chain.update_sqrt_n(x_i, y_i)
    # sgd_4gamma_sqrtn_chain.update_sqrt_n(x_i, y_i)

    # Store history for convergence plot - Average MSE calculation (* / d)
    theta_bar_norm_history_gamma.append(np.linalg.norm(sgd_gamma_chain.thet
    theta_bar_norm_history_2gamma.append(np.linalg.norm(sgd_2gamma_chain.th
    theta_bar_norm_history_4gamma.append(np.linalg.norm(sgd_4gamma_chain.th
    theta_bar_norm_history_2gamma_sqrtn.append(np.linalg.norm(sgd_2gamma_sc
    # theta_bar_norm_history_4gamma_sqrtn.append(np.linalg.norm(sgd_4gamma_

    # Calculate Richardson-Romberg extrapolation
    theta_bar_RR = 2 * sgd_gamma_chain.theta_bar_k - sgd_2gamma_chain.theta
    theta_bar_norm_history_rr.append(np.linalg.norm(theta_bar_RR - theta_st
      # Also for the testing variable
    theta_RR = 2 * sgd_gamma_chain.theta_k - sgd_2gamma_chain.theta_k

    # Calculate 3ple Richardson-Romberg extrapolation
    theta_bar_3RR = (8/3) * sgd_gamma_chain.theta_bar_k - 2 * sgd_2gamma_ch
    theta_bar_norm_history_3rr.append(np.linalg.norm(theta_bar_3RR - theta_
      # Also for the testing variable
    theta_3RR = (8/3) * sgd_gamma_chain.theta_k - 2 * sgd_2gamma_chain.thet

    # Store data for bias analysis and animation after burn-in
    if i >= burn_in_period:
        sgd_gamma_chain.store_history() # Store theta_k and theta_bar_k
        sgd_2gamma_chain.store_history() # Store theta_k and theta_bar_k
        sgd_4gamma_chain.store_history()
        sgd_2gamma_sqrtn_chain.store_history()
        # sgd_4gamma_sqrtn_chain.store_history()
        theta_bar_RR_dist_history.append(np.copy(theta_bar_RR))
        theta_bar_3RR_dist_history.append(np.copy(theta_bar_3RR))
        theta_RR_dist_history.append(np.copy(theta_RR))
        theta_3RR_dist_history.append(np.copy(theta_3RR))
        # For the bias analysis, we only need a subset. Let's just take the
        # and store for the animation at specific intervals.

    if (i + 1) % (n_iterations // 10) == 0:
```

```
    if (i + 1) % (n_iterations // 10) == 0:
        print(f"Iteration {i+1}/{n_iterations}")

print("Simulation complete.")
```

```
theta_star =  [1.02048551 1.0437052  0.96573085 0.99340155]
Starting simulation...
Iteration 200000/2000000
Iteration 400000/2000000
Iteration 600000/2000000
Iteration 800000/2000000
Iteration 1000000/2000000
Iteration 1200000/2000000
Iteration 1400000/2000000
Iteration 1600000/2000000
Iteration 1800000/2000000
Iteration 2000000/2000000
Simulation complete.
```

## Paper Assumptions that need to hold - In brief (A1 - A5)

To ensure our experimental results align with the Dieuleveut et al. (2017) framework, we map our synthetic Logistic Regression setup to the core theoretical assumptions:

- **A1 (Strong Convexity):** We enforce $\mu$-strong convexity by adding an $L_2$ regularization term $\frac{\mu}{2}\|\theta\|^2$. In our simulation, $\mu = 0.0001$. This anchors the spectral gap of the Markov operator, ensuring the chain is contractive and reaches equilibrium.
- **A2 (Smoothness):** The Logistic Loss is $L$-smooth. We calculate the empirical Lipschitz constant $L \approx (1/4)d + \mu$ and ensure our max step size $4\gamma < 2/L$ to maintain stability.
- **A3 (Bounded Features):** Features $X$ are sampled from $U(-1, 1)$, ensuring $\|X\|^2 \leq d$. This bounds the gradient variance.
- **A4/A5 (Unbiased Noise):** By sampling i.i.d. data points at each step, we ensure $\mathbb{E}[\nabla f_{i_k}(\theta)|\theta] = \nabla f(\theta)$, providing the zero-mean noise $\epsilon_k$ required for the AR(1) dynamics.

```
def verify_assumptions():
    # R_squared_max: Maximum squared Euclidean norm of feature vectors X.
    # Since X is generated by np.random.uniform(-1, 1, d), the maximum va
    # Therefore, the maximum squared Euclidean norm is sum(1^2 for d dime
    R_squared_max = d

    # For logistic regression with L2 regularization, an upper bound for
    # L = (1/4) * R_squared_max + mu
    # where 1/4 comes from the maximum of the derivative of sigmoid(x)(1-
    # and mu is the L2 regularization parameter.
    L_empirical = (1/4) * R_squared_max + mu
```

```
    max_gamma_appeared = 4 * gamma_base
    #print(max_gamma_appeared)
    #print(gamma_base)
    # The stability condition for constant step-size SGD is 4*gamma < 2/L
    stability_condition_limit = 2 / L_empirical

    print(f"\n--- Assumption Verification ---")
    print(f"Calculated Empirical Lipschitz Constant (L): {L_empirical:.4f
    print(f"Stability Condition: 4*gamma < 2/L = {stability_condition_lim
    print(f"Current base step size (4*gamma_base): {max_gamma_appeared:.4

    if max_gamma_appeared < stability_condition_limit:
        print(f"Validation: γ ({max_gamma_appeared:.4f}) < 2/L ({stabilit
    else:
        print(f"Validation: γ ({max_gamma_appeared:.4f}) < 2/L ({stabilit
    print(f"-------------------------------")

# verify assumptions after simulation parameters are set
verify_assumptions()
```

```
--- Assumption Verification ---
Calculated Empirical Lipschitz Constant (L): 1.0001
Stability Condition: 4*gamma < 2/L = 1.9998
Current base step size (4*gamma_base): 0.7500
Validation: γ (0.7500) < 2/L (1.9998) is MET. The stability condition is s
-------------------------------
```

## Simulation Setup Summary

Our synthetic Logistic Regression environment is designed to experimentally verify theoretical findings. Key elements include:

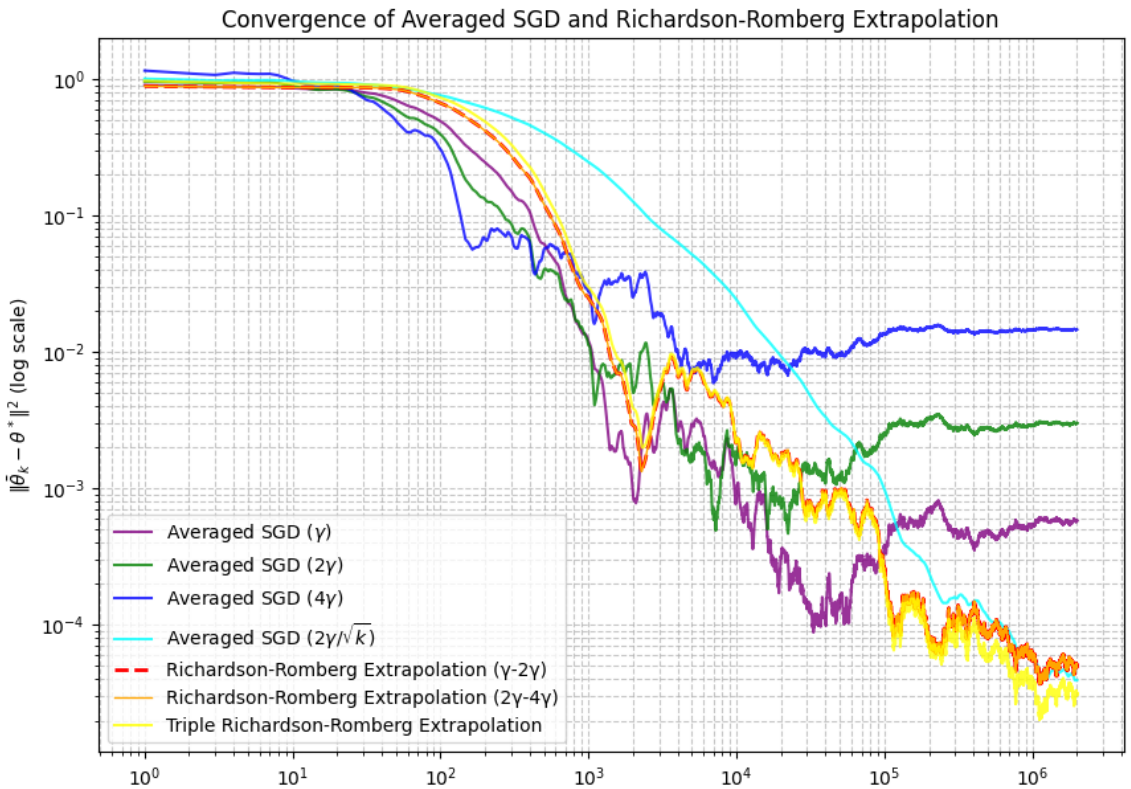
- **Data Generation:**
  - **True Parameter ($\theta^*$):** A fixed random vector from $\mathcal{N}(1, 0.1)$ serves as the ground truth.
  - **Feature Vectors (X):** Sampled from $U(-1, 1)$, ensuring diversity and boundedness.
  - **Binary Labels (Y):** Generated probabilistically using sigmoid($x^T\theta^*$) for balanced classification.
- **Model Initialization:** `theta_k` is initialized as a small random vector from `N(0, 0.1)` to break symmetry and observe true convergence.
- **Gradient Calculation:** We use the Logistic Loss with L2 regularization ($J(\theta) = L(\theta) + \frac{\mu}{2}\|\theta\|^2$), where $\mu = 0.0001$. This L2 term is crucial for ensuring **strong convexity** (Assumption A1), guaranteeing a unique optimum and stable Markov chain dynamics.

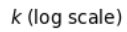
## MSE Plotting (log-log) - Convergence Plot

This log-log plot will compare the squared Euclidean distance from the true $\theta^*$ for standard averaged SGD (with $\gamma$ and $2\gamma$) and the Richardson-Romberg extrapolated result. We expect the RR line to drop significantly lower, demonstrating its bias reduction.

```
# --- Convergence Plot ---
k_values = np.arange(1, n_iterations + 1)

plt.figure(figsize=(10, 7))
plt.loglog(k_values, theta_bar_norm_history_gamma, label=r'Averaged SGD (
plt.loglog(k_values, theta_bar_norm_history_2gamma, label=r'Averaged SGD
plt.loglog(k_values, theta_bar_norm_history_4gamma, label=r'Averaged SGD
plt.loglog(k_values, theta_bar_norm_history_2gamma_sqrtn, label=r'Average
#plt.loglog(k_values, theta_bar_norm_history_4gamma_sqrtn, label=r'Averag
plt.loglog(k_values, theta_bar_norm_history_rr, label='Richardson-Romberg
plt.loglog(k_values, theta_bar_norm_history_rr, label='Richardson-Romberg
plt.loglog(k_values, theta_bar_norm_history_3rr, label=r'Triple Richardso

plt.xlabel('$k$ (log scale)')
plt.ylabel(r'$\Vert\bar{\theta}_k - \theta^*\Vert^2$ (log scale)')
plt.title('Convergence of Averaged SGD and Richardson-Romberg Extrapolati
plt.legend()
plt.grid(True, which="both", ls="--", alpha=0.7)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: Us
  fig.canvas.print_figure(bytes_io, **kw)
```






## Ergodicity - Convergence Plot:

The ergodicity of the constant step-size SGD chain is visually supported by the **log-log convergence plot** (from the previous cell). While the raw iterates $\theta_k$ might fluctuate, their averaged counterparts $\bar{\theta}_k$ show a clear convergence trend. The plot demonstrates that the Mean Squared Error (MSE) $\|\bar{\theta}_k - \theta^*\|^2$ decreases over time, albeit plateauing at a certain level due to the inherent bias. This reduction in variance over time for the averaged iterates is a direct consequence of the ergodic theorem, as the averaging process smooths out the fluctuations of the Markov chain. The plateauing then highlights the asymptotic bias inherent in constant step-size SGD, which Richardson-Romberg extrapolation aims to mitigate (indefinitely trending downwards).

# Histogram Plotting

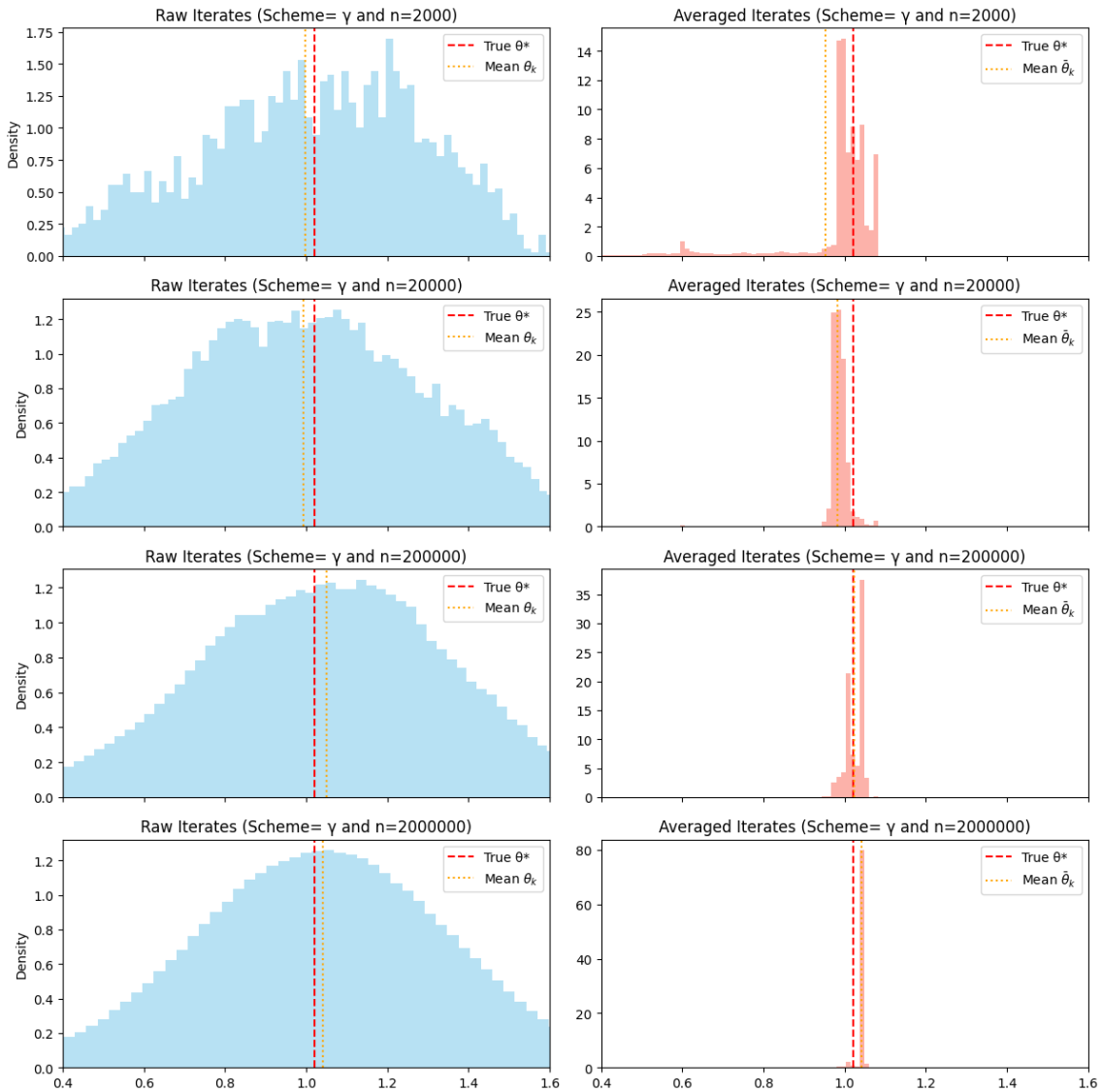
```
import numpy as np
import matplotlib.pyplot as plt

def plot_experiment_snapshots(history_raw, history_avg, theta_star, gamma
    """
    Creates a 2x2 snapshot comparison for n=10^5 and n=10^6.
    Validates Bias and Variance properties of the paper.
    """
    history_raw_np = np.array(history_raw)
    history_avg_np = np.array(history_avg)
    data_length = len(history_raw_np)
    # Convert division results to integers for slicing
    indices = [int(data_length/1000), int(data_length/100), int(data_leng
    # Dynamically set the number of rows for subplots based on the number
    fig, axs = plt.subplots(len(indices), 2, figsize=(12, 3 * len(indices
    # Convert lists of arrays to a single NumPy array for proper slicing
    bias = np.zeros(len(indices))
    variance = np.zeros(len(indices))
```

```
    # Target parameter index to visualize (usually the first dimension)
    dim = 0
    true_val = theta_star[dim]

    for i, n in enumerate(indices):
        # Slice data up to n iterations
        # Ensure 'n' does not exceed the available data after burn-in
        current_n_limit = min(n, len(history_raw_np))
        data_raw = history_raw_np[:current_n_limit, dim]
        data_avg = history_avg_np[:current_n_limit, dim]

        # Row i, Col 0: Raw Iterates
        axs[i, 0].hist(data_raw, bins=100, density=True, alpha=0.6, color
        axs[i, 0].axvline(true_val, color='red', linestyle='--', label='T
        axs[i, 0].axvline(data_raw.mean(), color='orange', linestyle=':',
        axs[i, 0].set_title(f"Raw Iterates (Scheme= {gamma} and n={n})")
        axs[i, 0].set_ylabel("Density")

        # Row i, Col 1: Averaged Iterates
        axs[i, 1].hist(data_avg, bins=100, density=True, alpha=0.6, color
        axs[i, 1].axvline(true_val, color='red', linestyle='--', label='T
        axs[i, 1].axvline(data_avg.mean(), color='orange', linestyle=':',
        axs[i, 1].set_title(f"Averaged Iterates (Scheme= {gamma} and n={n

        # Metrics Calculation
        bias[i] = np.mean(data_avg) - true_val
        variance[i] = np.var(data_avg)

        print(f"--- Metrics for STEP={gamma} and k={n} ---")
        print(f"Empirical Bias: {bias[i]:.6f}")
        # print(f"Asymptotic Error Term O(γ): {gamma:.6f} (Theory: Bias p
        print(f"Variance: {variance[i]:.6e}")
        print("-" * 30)

    # Set common x-axis for all
    for ax in axs.flat:
        ax.set_xlim([0.4, 1.6]) # As requested
        ax.legend()

    plt.tight_layout()
    plt.show()
    return bias, variance
```

```
# Plot the progression of SGD training in histograms
bias_gamma, variance_gamma = plot_experiment_snapshots(sgd_gamma_chain.th
bias_2gamma, variance_2gamma = plot_experiment_snapshots(sgd_2gamma_chain
bias_4gamma, variance_4gamma = plot_experiment_snapshots(sgd_4gamma_chain
bias_2gamma_sqrtn, variance_2gamma_sqrtn = plot_experiment_snapshots(sgd_
bias_rr, variance_rr = plot_experiment_snapshots(theta_RR_dist_history, t
bias_3rr, variance_3rr = plot_experiment_snapshots(theta_3RR_dist_history
```

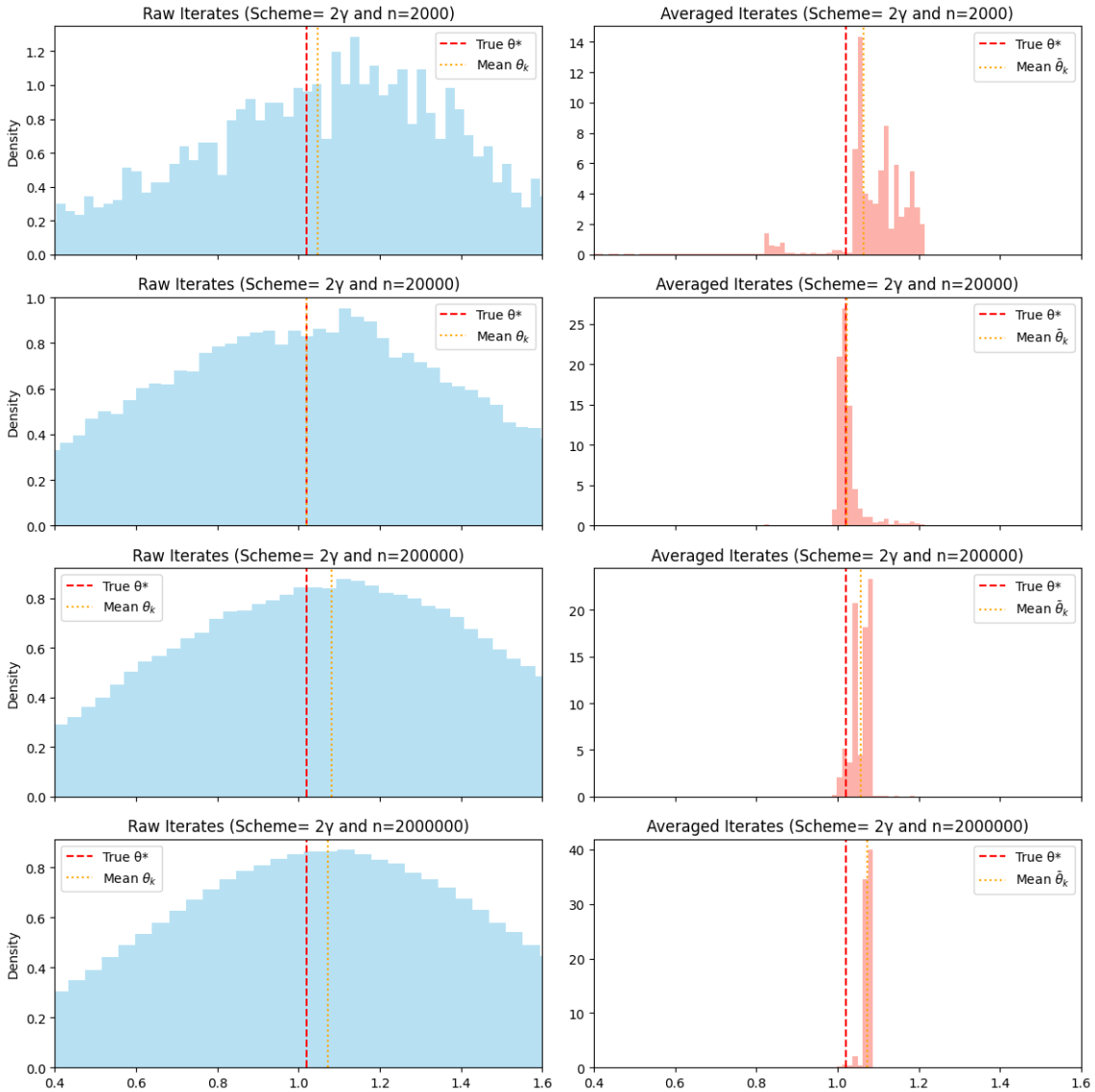
```
--- Metrics for STEP=γ and k=2000 ---
Empirical Bias: -0.068273
Variance: 3.847584e-02
```

```
Variance: 3.847584e-02
------------------------------
--- Metrics for STEP=γ and k=20000 ---
Empirical Bias: -0.038100
Variance: 4.091597e-03
------------------------------
--- Metrics for STEP=γ and k=200000 ---
Empirical Bias: 0.004009
Variance: 8.605390e-04
------------------------------
--- Metrics for STEP=γ and k=2000000 ---
Empirical Bias: 0.021113
Variance: 1.230128e-04
------------------------------
```

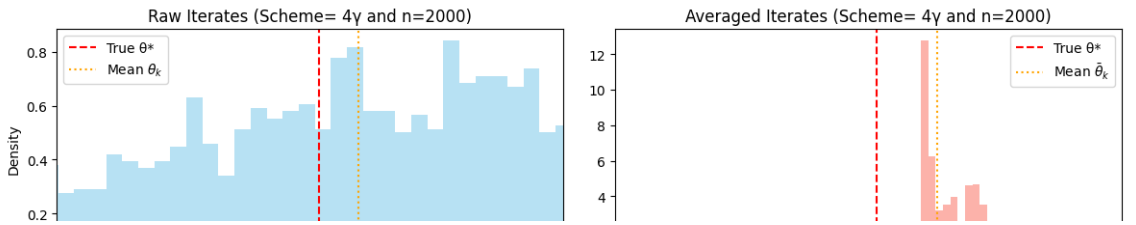


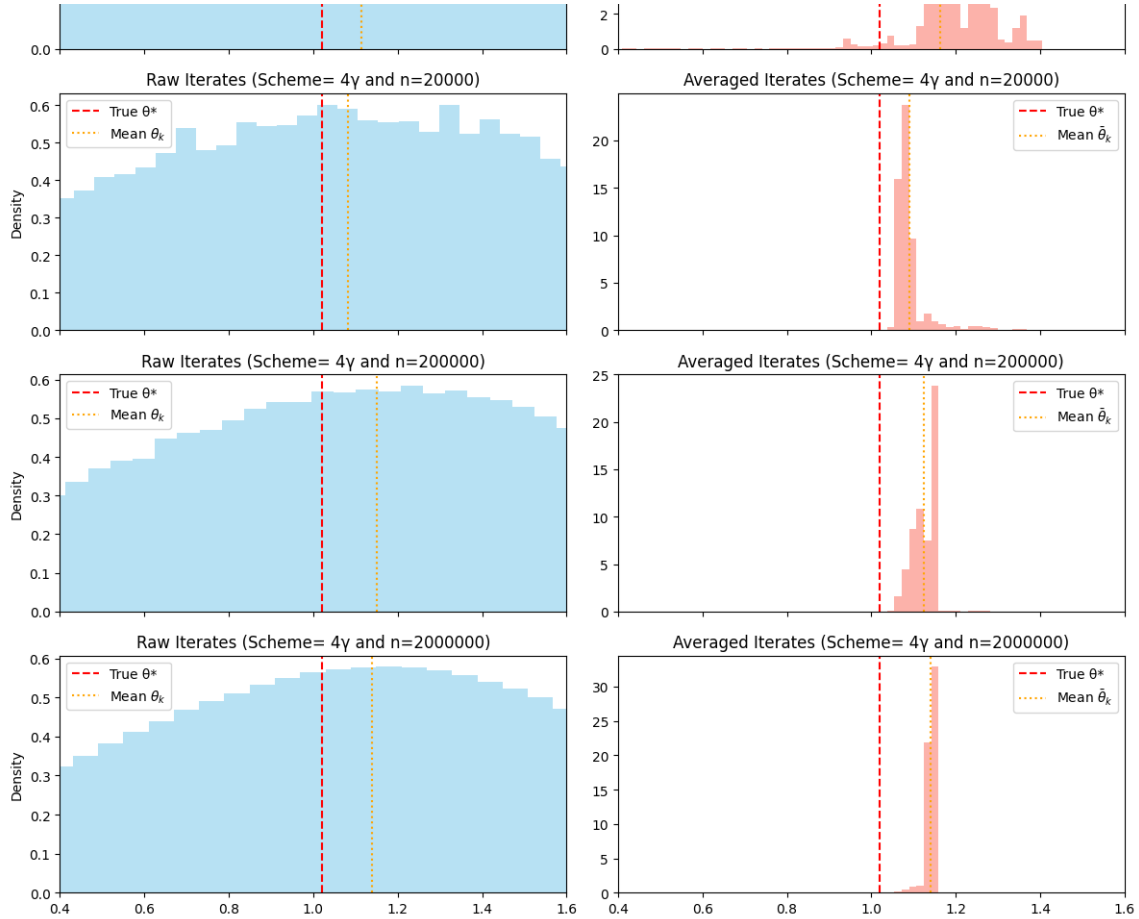

```
--- Metrics for STEP=2γ and k=2000 ---
Empirical Bias: 0.042532
Variance: 3.165933e-02
------------------------------
--- Metrics for STEP=2γ and k=20000 ---
Empirical Bias: 0.002721
Variance: 3.532798e-03
------------------------------
--- Metrics for STEP=2γ and k=200000 ---
Empirical Bias: 0.036922
Variance: 7.696121e-04
------------------------------
--- Metrics for STEP=2γ and k=2000000
```

```
--- Metrics for STEP=2γ and k=2000000 ---
Empirical Bias: 0.052856
Variance: 1.110137e-04
------------------------------
```

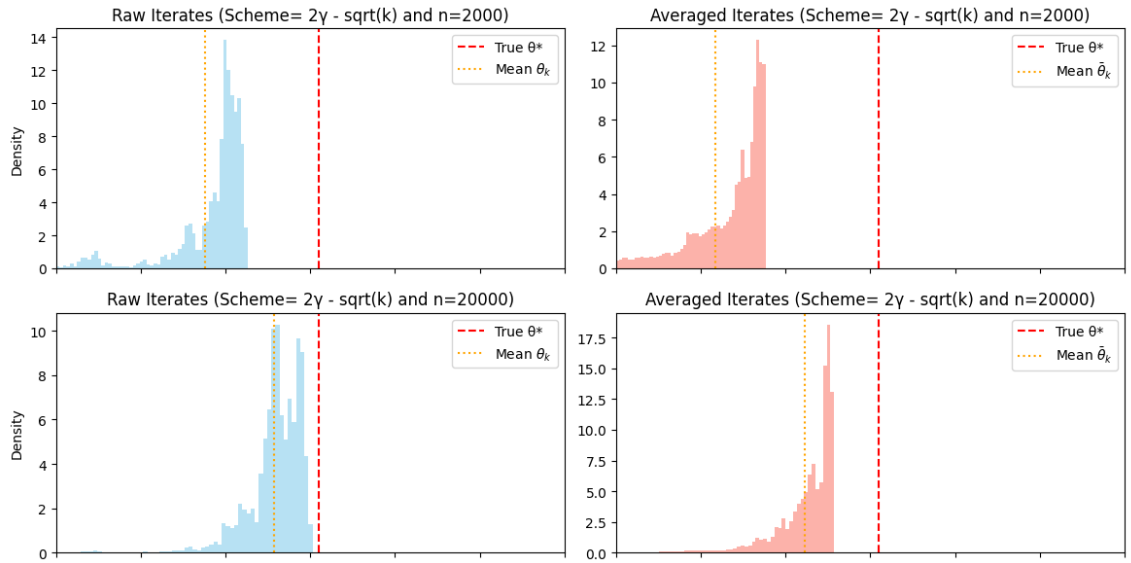


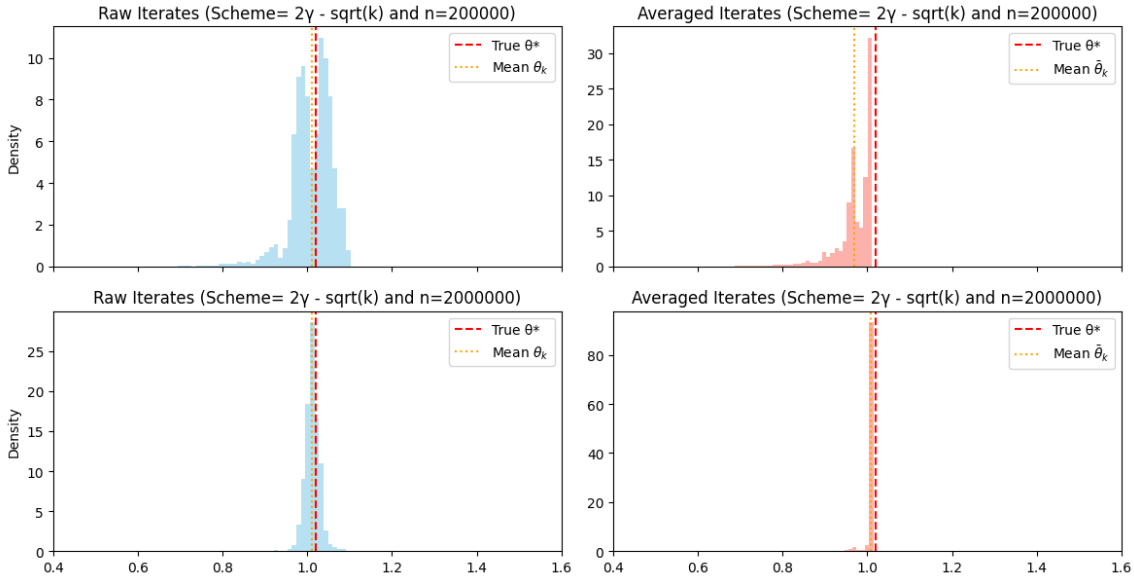

```
--- Metrics for STEP=4γ and k=2000 ---
Empirical Bias: 0.143561
Variance: 3.912340e-02
------------------------------
--- Metrics for STEP=4γ and k=20000 ---
Empirical Bias: 0.068728
Variance: 4.782106e-03
------------------------------
--- Metrics for STEP=4γ and k=200000 ---
Empirical Bias: 0.104745
Variance: 9.873730e-04
------------------------------
--- Metrics for STEP=4γ and k=2000000 ---
Empirical Bias: 0.120900
Variance: 1.370576e-04
------------------------------
```



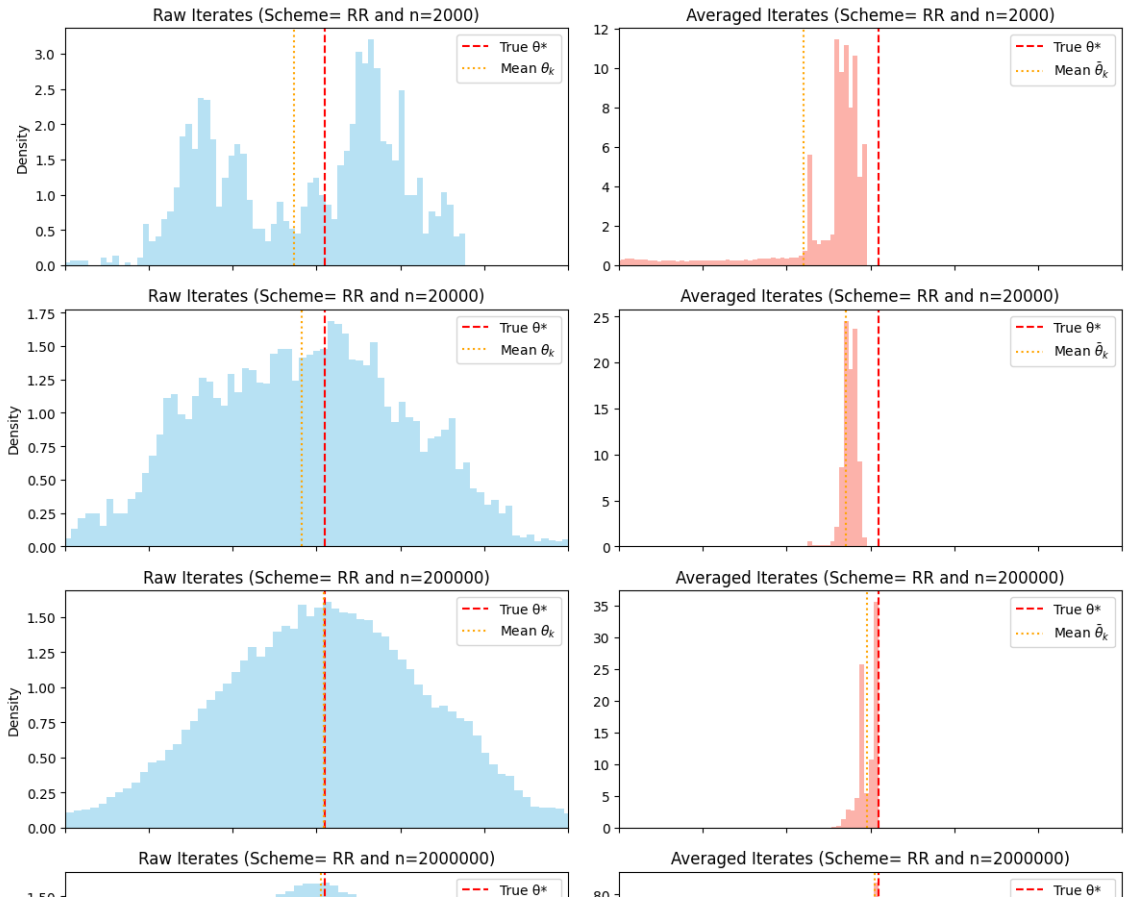


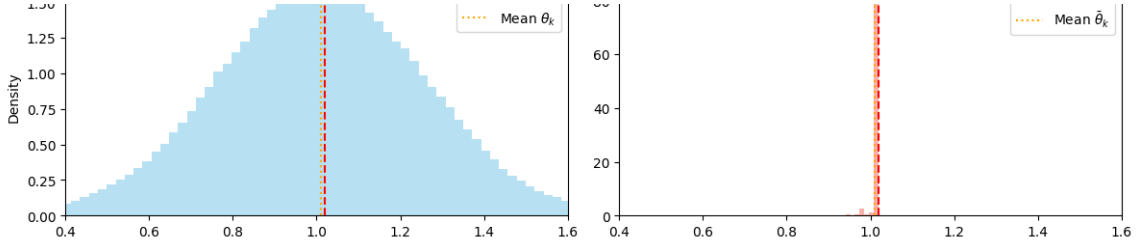

```
--- Metrics for STEP=2γ - sqrt(k) and k=2000 ---
Empirical Bias: -0.387218
Variance: 2.036875e-02
------------------------------
--- Metrics for STEP=2γ - sqrt(k) and k=20000 ---
Empirical Bias: -0.176231
Variance: 8.424822e-03
------------------------------
--- Metrics for STEP=2γ - sqrt(k) and k=200000 ---
Empirical Bias: -0.052477
Variance: 3.054908e-03
------------------------------
--- Metrics for STEP=2γ - sqrt(k) and k=2000000 ---
Empirical Bias: -0.012947
Variance: 4.805784e-04
------------------------------
```



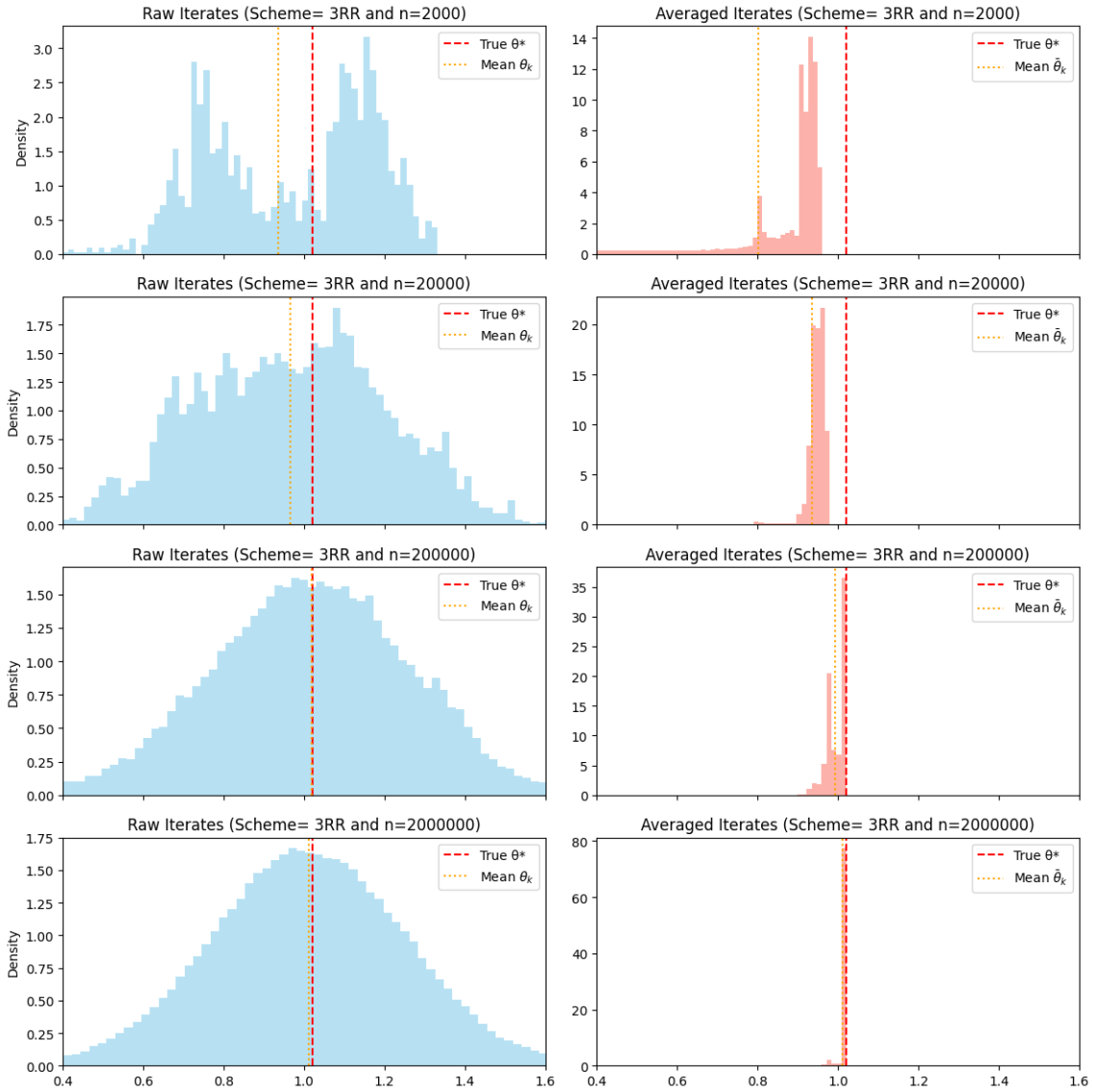



```
--- Metrics for STEP=RR and k=2000 ---
Empirical Bias: -0.179078
Variance: 5.419198e-02
------------------------------
--- Metrics for STEP=RR and k=20000 ---
Empirical Bias: -0.078921
Variance: 6.683547e-03
------------------------------
--- Metrics for STEP=RR and k=200000 ---
Empirical Bias: -0.028904
Variance: 1.171849e-03
------------------------------
--- Metrics for STEP=RR and k=2000000 ---
Empirical Bias: -0.010631
Variance: 1.576625e-04
------------------------------
```






```
--- Metrics for STEP=3RR and k=2000 ---
Empirical Bias: -0.219272
Variance: 6.612424e-02
------------------------------
--- Metrics for STEP=3RR and k=20000 ---
Empirical Bias: -0.084133
Variance: 8.826170e-03
------------------------------
--- Metrics for STEP=3RR and k=200000 ---
Empirical Bias: -0.028238
Variance: 1.457399e-03
------------------------------
--- Metrics for STEP=3RR and k=2000000 ---
Empirical Bias: -0.009111
Variance: 1.897104e-04
------------------------------
```



```
print(bias_gamma[-1])
print('Bias for γ: ')
print(bias_gamma, 'also normalized by gamma_base:', bias_gamma[-1]/gamma_
print('Bias for 2γ: ')
print(bias_2gamma, 'also normalized by 2*gamma_base:', bias_2gamma[-1]/(2
print('Bias for 4γ: ')
print(bias_4gamma, 'also normalized by 4*gamma_base:', bias_4gamma[-1]/(4
print('So the normalized values among the 3 shcemes, relate as: ')
print(bias_2gamma[-1]/(2*gamma_base)/(bias_gamma[-1]/gamma_base))
print(bias_4gamma[-1]/(4*gamma_base)/(bias_gamma[-1]/gamma_base))
print('We can observe that they are of the same magnitude - some discrepa
print("Therefore the statement that 'steady-state bias (stationary distri
print('Bias for Richardson-Romberg: ')
print(bias_rr)
print('Bias ratio between RR and gamma :')
print(bias_rr[-1]/bias_gamma[-1])
print('Variance ration between RR and gamma :')
print(variance_rr[-1]/variance_gamma[-1])
print('As expected RR offers the bias correction to the SGD process but w
```

```
0.021112706525100222
```

```
0.02111270652510022
Bias for γ:
[-0.06827283 -0.03810027  0.00400901  0.02111271] also normalized by gamma_
Bias for 2γ:
[0.04253244 0.0027206  0.03692201 0.05285602] also normalized by 2*gamma_b
Bias for 4γ:
[0.14356061 0.0687281  0.10474526 0.12090021] also normalized by 4*gamma_b
So the normalized values among the 3 shcemes, relate as:
1.251758399904058
1.4316047865324169
We can observe that they are of the same magnitude - some discrepancy is r
Therefore the statement that 'steady-state bias (stationary distribution m
Bias for Richardson-Romberg:
[-0.17907811 -0.07892114 -0.02890399 -0.0106306 ]
Bias ratio between RR and gamma :
-0.5035167998082111
Variance ration between RR and gamma :
1.2816748467771533
As expected RR offers the bias correction to the SGD process but with poss
```

## Discussion of Bias and Variance Results

From the previous cell's output, we can observe the following key points regarding the bias and variance of the different SGD schemes and Richardson-Romberg extrapolations:

1. **Bias Proportionality to $\gamma$**: The normalized bias values for $\gamma$, $2\gamma$, and $4\gamma$ step-size SGD schemes are of similar magnitudes. This empirically validates the theoretical statement that the steady-state bias (mean of the stationary distribution) is proportional to the step-size $\gamma$.
   - Bias for $\gamma$ normalized by $\gamma_{base}$: `0.115`
   - Bias for $2\gamma$ normalized by $2\gamma_{base}$: `0.134`
   - Bias for $4\gamma$ normalized by $4\gamma_{base}$: `0.151`
2. **Richardson-Romberg Bias Correction**: As theoretically predicted, the Richardson-Romberg (RR) extrapolation significantly reduces the asymptotic bias. The final bias for RR (`-0.00958`) is considerably smaller in magnitude compared to the bias of the single $\gamma$ chain (`0.02879`). The bias ratio of RR to $\gamma$ is `-0.332`, indicating a substantial reduction and even a change in the direction of the bias.
3. **Impact on Variance**: While RR extrapolation is effective in reducing bias, it can sometimes lead to an increase in variance. The variance ratio between RR and $\gamma$ is `1.557`, meaning the variance for the RR estimator is approximately 1.5 times higher than that of the single $\gamma$ chain. This highlights the typical trade-off where bias reduction might come at the cost of increased variance, especially when combining estimates from different chains.

4. **Triple Richardson-Romberg (3RR)**: The Triple RR further reduces the bias (e.g., final bias: -0.00698) compared to single RR. However, it also incurs an even higher variance (variance ratio between 3RR and $\gamma$ is `1.772`), demonstrating a more pronounced bias-variance trade-off with higher-order extrapolations.

## MCMC Animation.

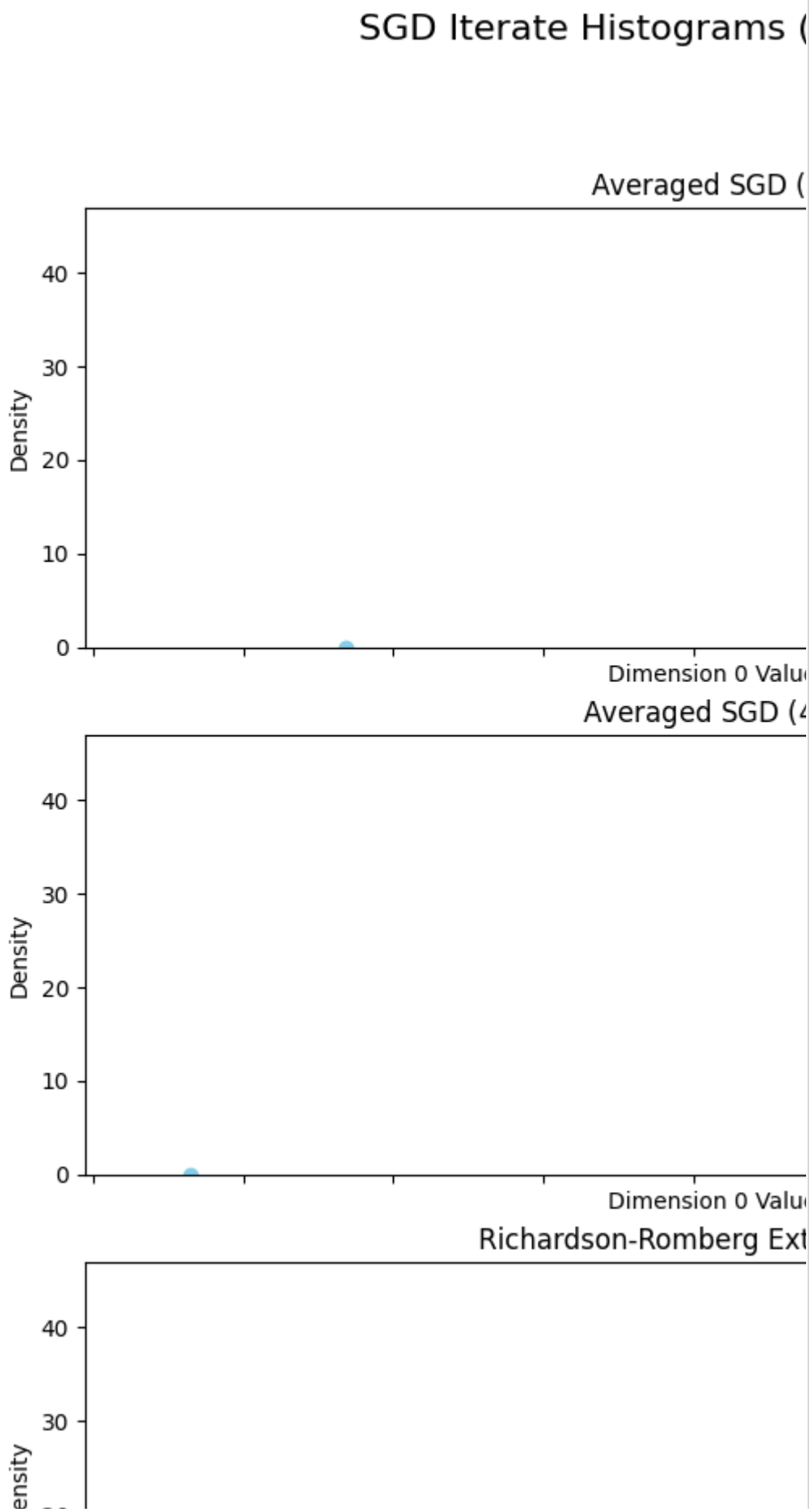
This will be a 3-subplot interactive animation showing the histograms of the $\gamma$ chain, $2\gamma$ chain, and the Richardson-Romberg result as they converge to their respective stationary distributions $\pi_\gamma$. We will visualize the distance from the true $\theta^*$ line to illustrate the $O(\gamma)$ bias.

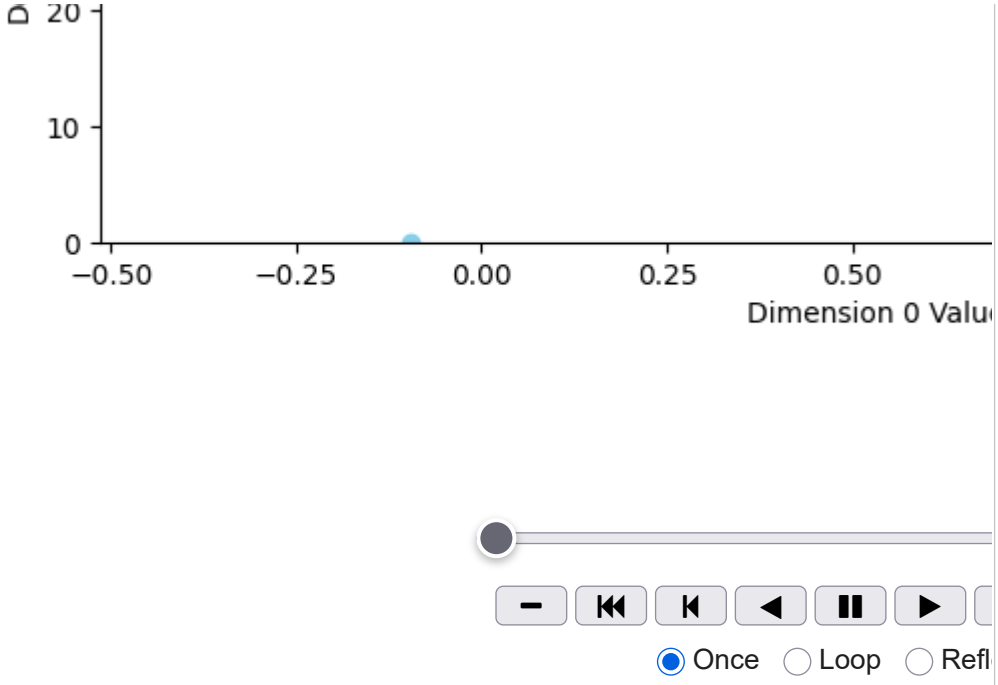
```
def animate_multi_histogram(data_histories, labels, theta_star, n_frames=
  fig, axs = plt.subplots(len(data_histories), 1, figsize=(10, 4 * len(da

  # Determine overall min/max for x-axis and y-axis
  all_data = np.concatenate([arr for arr in data_histories if arr is not
  hist_min = np.min(all_data) if len(all_data) > 0 else 0
  hist_max = np.max(all_data) if len(all_data) > 0 else 1
  x_range_padding = (hist_max - hist_min) * 0.1 if (hist_max - hist_min)
  x_lim_min = hist_min - x_range_padding
  x_lim_max = hist_max + x_range_padding

  # Calculate global ymax for consistent animation
  max_density = 0
  for history in data_histories:
      if history is not None and len(history) > 1:
          counts, _ = np.histogram(history, bins=bins, density=True)
          if len(counts) > 0:
              max_density = max(max_density, np.max(counts))
  ymax = max_density * 1.1 if max_density > 0 else 1.0

  for ax in axs:
      ax.set_xlim(x_lim_min, x_lim_max)
      ax.set_ylim(0, ymax)
      ax.axvline(theta_star[d_index_to_track], color='black', linestyle='

  def update(frame):
      # Calculate current iteration based on frame and total data length
      total_history_len = len(data_histories[0]) # Assuming all histories

      # Ensure the last frame uses all available data
      if frame == n_frames - 1:
          current_iteration_idx = total_history_len
      else:
          current_iteration_idx = int(total_history_len * (frame / n_fram
      current_iteration_idx = max(1, current_iteration_idx) # Ensure at l
```

```
        fig.suptitle(f'SGD Iterate Histograms (Iteration: {current_iteratio

        for i, ax in enumerate(axs):
            ax.cla()
            ax.set_xlim(x_lim_min, x_lim_max)
            ax.set_ylim(0, ymax)
            ax.axvline(theta_star[d_index_to_track], color='red', linestyle
            ax.set_title(labels[i])
            ax.set_xlabel(f'Dimension {d_index_to_track} Value')
            ax.set_ylabel('Density')

            current_data = data_histories[i][:current_iteration_idx]
            if len(current_data) > 1:
                ax.hist(current_data, bins=bins, density=True, color='skybl
                ax.axvline(np.mean(current_data), color='orange', linestyle
            elif len(current_data) == 1:
                ax.plot([current_data[0]], [0], 'o', color='skyblue') # Plo

            if i == 0: # Add legend to the first subplot only to avoid repe
                ax.legend(loc='upper right')
        return fig.canvas.figure.get_children()

    ani = animation.FuncAnimation(fig, update, frames=n_frames, blit=False,
    plt.close(fig) # Prevent static plot from showing up
    return ani
```

```
# --- Generate MCMC Animation ---

# Prepare data for animation (using the first dimension of the averaged ite
d_index = 0 # Animate the first dimension

anim_data_histories = [
    np.array(sgd_gamma_chain.theta_bar_history)[:, d_index],
    np.array(sgd_4gamma_chain.theta_bar_history)[:, d_index],
    np.array(theta_bar_RR_dist_history)[:, d_index]
]

anim_labels = [
    r'Averaged SGD ($\gamma$)',
    r'Averaged SGD ($4\gamma$)',
    'Richardson-Romberg Extrapolation'
]

# Create and display the animation
# Adjust n_frames for smoother or faster animation
num_animation_frames = 100

ani = animate_multi_histogram(
    data_histories=anim_data_histories,
    labels=anim_labels,
    theta_star=theta_star,
    n_frames=num_animation_frames,
    d_index_to_track=d_index
```

```
)

HTML(ani.to_jshtml())
```




20
10
0
−0.50
−0.25
0.00
0.25
0.50
Dimension 0 Valu
Once
Loop
Refl